Reviewer Pack — Vertex Star-Discrepancy Equals Spectral Norm for XOR Comm

Entry d27e7e45b29b · INCONCLUSIVE

SEC autonomous research engine — attribution: Ludovico Kubler

2026-05-17 02:29:48 UTC

Vertex Star-Discrepancy Equals Spectral Norm for XOR Comm

Entry ID: `d27e7e45b29b` **Verdict**: `INCONCLUSIVE` **Recorded**: 2026-05-17 02:29:48 UTC

1. Conjecture

Field A (mathematical branch): Geometric discrepancy theory (vertex-restricted L∞ star discrepancy of {0,1}^n point sets, in the Chazelle–Matousek tradition of axis-parallel box discrepancy) **Field B** (complexity object): Randomized two-party communication complexity of XOR-functions f(x,y)=g(x⊕y) accessed through the discrepancy-method lower bound disc(M_f)=||ĝ||_∞

Statement:

> For any non-constant g:{0,1}^n→{−1,+1}, let P_g={z∈{0,1}^n : g(z)=−1} and define the lattice star discrepancy D*lat(g) := max{a∈{0,1}^n} | |P_g ∩ [0,a]⊓|/2^n − (|P_g|/2^n)·(2{popcount(a)}/2^n) |, where [0,a]⊓={z∈{0,1}^n : z≤a coordinatewise}; let ||ĝ||∞ := max{S⊆[n]} |ĝ(S)| be the spectral norm. Conjecture: there exist absolute constants c_1=1/8 and c_2=8 such that for every n≥4 and every non-constant g, c_1·||ĝ||_∞ ≤ D*lat(g) ≤ c_2·||ĝ||∞·log_2(n+1). A single (n,g) pair violating either inequality refutes the conjecture.

Rationale (proposer's reasoning):

> The discrepancy lower bound on randomized XOR-communication is exactly log(1/||ĝ||∞), yet ||ĝ||∞ is a global Fourier quantity with no geometric face. Vertex star discrepancy is the canonical Chazelle–Matousek irregularity measure of a point set inside [0,1]^n; restricting test boxes to lattice corners makes it computable in O(n·2^n) via the zeta transform. If D*lat ≈ ||ĝ||∞, this gives the first concrete geometric (axis-parallel box) certificate of XOR-communication hardness, opening a Banaszczyk/Kashin-style geometric attack on unsaturated R(DISJ)-like bounds.

Taxonomy category: `DISPERSION_DISCREPANCY` (status at proposal time: alive)

2. Pre-registration (Popper-style)

Hash (SHA-256 prefix): `2d2bdbdaa37ccae1`

This hash commits to the conjecture statement, acceptance criterion, and seed list **before** the empirical test runs. Tampering with the test or its data after this hash was computed would be detectable.

Acceptance criterion:

Across all 210 trials (7 values of n × 30 seeds × diverse g families), both inequalities $0.125 \cdot ||\hat{g}||_\infty \le Dlat(g)$ *and* $Dlat(g) \le 8 \cdot ||\hat{g}||_\infty \cdot \log_2(n+1)$ must hold; report min lower-ratio ($D_{lat}/||\hat{g}||_\infty$) and max upper-ratio ($D_{lat}/(||\hat{g}||_\infty \cdot \log_2(n+1))$).

3. Barrier filter (F1)

Final: PASS

Barrier	Verdict	Confidence	LLM_1	LLM_2
RELATIVIZATION	SAFE	0.92	SAFE	SAFE
NATURAL_PROOFS	UNCERTAIN	0.95	SAFE	HITS
ALGEBRIZATION	SAFE	0.95	SAFE	SAFE
KARP_LIPTON	SAFE	0.95	SAFE	SAFE

4. Novelty audit

Verdict: NOVEL against 2 hits across arXiv + Semantic Scholar + ECCC

Search queries (3): - `star discrepancy Boolean cube Fourier spectral norm` - `XOR functions communication complexity discrepancy method spectral norm` - `axis-parallel box discrepancy hypercube Walsh Fourier coefficients`

Top relevant hits considered: - [s2:1704.02537] Dual polynomials and communication complexity of XOR functions - [s2:19311e592db3bd54ad8685ae0d27b815c87bef9c] Dual polynomials and communication complexity of XOR functions

5. Empirical test harness

Multi-seed protocol: 5 seeds (11, 23, 37, 53, 71)
Sandbox: isolated subprocess, stdlib-only, ≤90s wall time per cycle **Execution**: rc=124, elapsed=240.0s

5.1 Generated Python source

```python
import sys
import random
import math
import itertools
from collections import defaultdict

def generate_random_g(n, seed):
    random.seed(seed)
    g = [random.choice([-1, 1]) for _ in range(2**n)]
    return g

def generate_parity_g(n, seed):
    random.seed(seed)
    S = random.sample(range(n), random.randint(1, n))
    g = [1] * (2**n)
    for i in range(2**n):
        z = [int(b) for b in bin(i)[2:].zfill(n)]
        if sum(z[j] for j in S) % 2 == 1:
            g[i] = -1
    return g

def generate_junta_g(n, seed):
```

```
    random.seed(seed)
    k = math.ceil(n / 2)
    S = random.sample(range(n), k)
    g = [1] * (2**n)
    for i in range(2**n):
        z = [int(b) for b in bin(i)[2:].zfill(n)]
        if sum(z[j] for j in S) >= k / 2:
            g[i] = -1
    return g

def generate_and_g(n):
    g = [1] * (2**n)
    for i in range(2**n):
        z = [int(b) for b in bin(i)[2:].zfill(n)]
        if all(z[j] == 1 for j in range(n)):
            g[i] = -1
    return g

def generate_or_g(n):
    g = [1] * (2**n)
    for i in range(2**n):
        z = [int(b) for b in bin(i)[2:].zfill(n)]
        if any(z[j] == 1 for j in range(n)):
            g[i] = -1
    return g

def generate_maj_g(n):
    g = [1] * (2**n)
    for i in range(2**n):
        z = [int(b) for b in bin(i)[2:].zfill(n)]
        if sum(z[j] for j in range(n)) >= n / 2:
            g[i] = -1
    return g

def walsh_hadamard_transform(g, n):
    gh = [0.0] * (2**n)
    for S in range(2**n):
        for z in range(2**n):
            dot_product = sum((int(b) for b in bin(S & z)[2:].zfill(n)))
            gh[S] += g[z] * ((-1) ** dot_product)
    return gh

def spectral_norm(gh, n):
    return max(abs(x) for x in gh) / (2**n)

def star_discrepancy(g, n):
    c = [0] * (2**n)
    for a in range(2**n):
        for z in range(2**n):
            if all((z & (1 << i)) <= (a & (1 << i)) for i in range(n)):
                if g[z] == -1:
                    c[a] += 1
    P_g = sum(1 for x in g if x == -1)
    discrepancy = 0.0
    for a in range(2**n):
        popcount = bin(a).count('1')
```

```
            term = abs(c[a] / (2**n) - (P_g / (2**n)) * (2**popcount / (2**n)))
            if term > discrepancy:
                discrepancy = term
    return discrepancy

def run_trial(seed):
    n_values = [6, 8, 10, 12, 14, 16, 18]
    n = random.choice(n_values)
    random.seed(seed)
    g_type = random.choice(['random', 'parity', 'junta', 'and', 'or', 'maj'])
    if g_type == 'random':
        g = generate_random_g(n, seed)
    elif g_type == 'parity':
        g = generate_parity_g(n, seed)
    elif g_type == 'junta':
        g = generate_junta_g(n, seed)
    elif g_type == 'and':
        g = generate_and_g(n)
    elif g_type == 'or':
        g = generate_or_g(n)
    elif g_type == 'maj':
        g = generate_maj_g(n)

    gh = walsh_hadamard_transform(g, n)
    spectral_norm_val = spectral_norm(gh, n)
    discrepancy = star_discrepancy(g, n)

    lower_bound = 0.125 * spectral_norm_val
    upper_bound = 8 * spectral_norm_val * math.log2(n + 1)

    conjecture_holds = (lower_bound <= discrepancy <= upper_bound)
    counterexample = ""
    if not conjecture_holds:
        if discrepancy < lower_bound:
            counterexample = f"Lower bound violated: D*_lat={discrepancy} < {lower_bound}"
        else:
            counterexample = f"Upper bound violated: D*_lat={discrepancy} > {upper_bound}"

    return {
        "metric_name": "star_discrepancy",
        "metric_value": discrepancy,
        "instances_tested": 1,
        "conjecture_holds": conjecture_holds,
        "counterexample": counterexample,
        "spectral_norm": spectral_norm_val,
        "n": n,
        "g_type": g_type
    }

if __name__ == "__main__":
    seeds = [int(arg) for arg in sys.argv[1:]] if len(sys.argv) > 1 else [2, 3, 5, 7, 11, 13, 17,
    results = []
    for seed in seeds:
        trial = run_trial(seed)
        print(f"TRIAL: {trial}")
        results.append(trial)
```

```
metric_values = [trial["metric_value"] for trial in results]
mean_metric = sum(metric_values) / len(metric_values)
std_metric = math.sqrt(sum((x - mean_metric) ** 2 for x in metric_values) / len(metric_values)
support_fraction = sum(1 for trial in results if trial["conjecture_holds"]) / len(results)

if all(trial["conjecture_holds"] for trial in results):
    print(f"RESULT: SUPPORTED mean={mean_metric} std={std_metric} support_fraction={support_fr
elif any(not trial["conjecture_holds"] for trial in results):
    first_failing_seed = next(trial["seed"] for trial in results if not trial["conjecture_hold
    counterexample = next(trial["counterexample"] for trial in results if not trial["conjectur
    print(f"RESULT: FALSIFIED counterexample=\"{counterexample}\" first_failing_seed={first_fa
else:
    print("RESULT: INCONCLUSIVE reason=unknown")
```

6. Per-seed results

(no seed-level data — test crashed before producing TRIAL: lines)

7. Test stdout (last 2KB)

```
TIMEOUT after 240s
```

8. Critic adversarial review

Critic verdict: CHALLENGE

Critic reasoning:

> skipped: test crashed before producing data

9. Final verdict & safety rail

Verdict: INCONCLUSIVE

Reasoning:

> The test timed out after 240s (returncode=124) without producing a RESULT line or any trial data, so neither the support nor falsification condition can be evaluated. | next: Reduce the maximum n or restrict the enumeration over a∈{0,1}^n (e.g., sample or use a smarter combinatorial bound) so the 210 trials complete within the time budget.

11. Audit log (LLM calls)

Total LLM calls: 9

#	Phase	Provider	Model	Tokens in	out	Latency (ms)
1	propose	claude_max	opus	0	0	109237
2	preregistration	claude_max	opus	0	0	5378
3	novelty	claude_max	opus	0	0	3151
4	novelty	claude_max	opus	0	0	8006
5	test_gen	mistral	codestral-latest	0	0	312600
6	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	269599
7	test_gen	mistral	codestral-latest	0	0	308521
8	test_gen	mistral	codestral-latest	0	0	310525
9	judge	claude_max	opus	0	0	9075

Totals: 0 input tokens, 0 output tokens, ~$0.0000 cost, 1336091 ms total latency. Provider mix: {'claude_max': 5, 'mistral': 3, 'ollama_remote': 1}

(full prompt+response transcripts available in `research/audit/d27e7e45b29b.jsonl`)

12. Reproducibility

To reproduce this cycle:

1. Apply the test harness in section 5.1 with seeds 11,23,37,53,71
2. The Python sandbox is pure-stdlib; any Python 3.11+ should suffice
3. The full LLM transcripts are in `research/audit/d27e7e45b29b.jsonl` for verification of provenance
4. The pre-registration hash in section 2 commits to the test design before execution; tampering would be detectable.

Sandbox file: `research/pvsnp_sandbox/test_*.py` (per-cycle)
Replay tarball: `research/replay/d27e7e45b29b.tar.gz` (if generated)